

# **VISVESVARAYA TECHNOLOGICAL UNIVERSITY**

“JnanaSangama”, Belgaum -590014, Karnataka.



## **LAB REPORT on**

## **Artificial Intelligence (23CS5PCAIN)**

*Submitted by*

**Shashank Gowda L (1BM23CS311)**

*in partial fulfillment for the award of the degree of*  
**BACHELOR OF ENGINEERING**  
*in*  
**COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING**

(Autonomous Institution under VTU)

**BENGALURU-560019**

**Aug 2025 to Dec 2025**

**B.M.S. College of Engineering,**  
**Bull Temple Road, Bangalore 560019**  
(Affiliated To Visvesvaraya Technological University, Belgaum)  
**Department of Computer Science and Engineering**



**CERTIFICATE**

This is to certify that the Lab work entitled “Artificial Intelligence (23CS5PCAIN)” carried out by **Shashank Gowda L (1BM23CS311)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

|                                                                              |                                                                  |
|------------------------------------------------------------------------------|------------------------------------------------------------------|
| Lab faculty Incharge Name<br>Assistant Professor<br>Department of CSE, BMSCE | Dr. Kavitha Sooda<br>Professor & HOD<br>Department of CSE, BMSCE |
|------------------------------------------------------------------------------|------------------------------------------------------------------|

## Index

| Sl. No. | Date       | Experiment Title                                                                                                      | Page No. |
|---------|------------|-----------------------------------------------------------------------------------------------------------------------|----------|
| 1       | 30-9-2024  | Implement Tic –Tac –Toe Game<br>Implement vacuum cleaner agent                                                        | 5-7      |
| 2       | 7-10-2024  | Implement 8 puzzle problems using Depth First Search (DFS)<br>Implement Iterative deepening search algorithm          | 8-11     |
| 3       | 14-10-2024 | Implement A* search algorithm                                                                                         | 12-15    |
| 4       | 21-10-2024 | Implement Hill Climbing search algorithm to solve N-Queens problem                                                    | 16-17    |
| 5       | 28-10-2024 | Simulated Annealing to Solve 8-Queens problem                                                                         | 18-19    |
| 6       | 11-11-2024 | Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.    | 20-21    |
| 7       | 2-12-2024  | Implement unification in first order logic                                                                            | 22-24    |
| 8       | 2-12-2024  | Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning. | 25-26    |
| 9       | 16-12-2024 | Create a knowledge base consisting of first order logic statements and prove the given query using Resolution         | 27-28    |
| 10      | 16-12-2024 | Implement Alpha-Beta Pruning.                                                                                         | 29-30    |

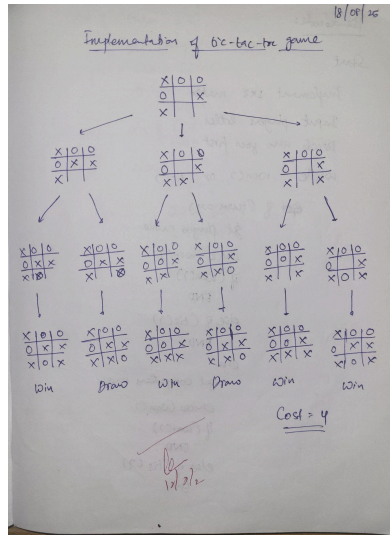
Github Link: <https://github.com/ShashankGowdaL69/AIILab311>



## Program 1

Implement Tic-Tac-Toe Game.

Implement a vacuum cleaner agent.



Pseudocode:

Start

Implement 3x3 matrix

Input players letters

Decide who goes first

while !won() or tie()

if (turn == 1)

get player's move

check won()

if (won())

END

else if (tie())

END

else

get computer's move

check won()

if (won())

END

else if (tie())

END

END

do if (turn == 0)

get computer's move

check won()

if (won())

END

else if (tie())

END

else

get player's move

check won()

if (won())

END

else if (tie())

END

END

Output:

Player X

Enter row: 1

Enter column: 1

Player O

Enter row: 0

Enter column: 0

Player X

Enter row: 2

Enter column: 2

Player O

Enter row: 0

Enter column: 1

Player X

Enter row: 2

Enter column: 2

Player X wins!

Implementation of vacuum cleaning agent

Algorithm for 2 rooms:

s1: Start

s2: Ask user for initial room

s3: If A:

room = A

Dirty = true

If B:

room = B

Dirty = true

s4: If room = A:

Ask user for action

If A:

move

room = B

else if C:

room = A

else if C and Dirty = true

Dirty = false

else print "Invalid action"

s5: If room = B:

Ask user for action

If B:

room = A

else if C:

room = B

END

else if C and Dirty = true

Dirty = false

else print "Invalid action"

s6: Repeat s4 and s5 until Dirty = false

s7: End

Output:

Enter starting room (A/B): 1

Current room: A

Room A dirty? True

Room B dirty? True

Enter action: suck

Sucking dirt in room A

Current room: A

Room A dirty? False

Room B dirty? True

Enter action: move

Moving robot to room B

Current room: B

Room A dirty? False

Room B dirty? True

Enter action: suck

Sucking dirt in room B

Both rooms are clean

```
print("ShashankGowdaL 1BM23CS311")
```

```
def print_board(board):
```

```
    for row in board:
```

```
        print(" ".join(row))
```

```
    print("-" * 5)
```

```
def check_win(board, player):
```

```
    for row in board:
```

```
        if all([cell == player for cell in row]):
```

```
            return True
```

```
    for col in range(3):
```

```
        if all([board[row][col] == player for row in range(3)]):
```

```
            return True
```

```

    if board[0][0] == board[1][1] == board[2][2] == player:
        return True
    if board[0][2] == board[1][1] == board[2][0] == player:
        return True
    return False

def check_draw(board):
    for row in board:
        if " " in row:
            return False
    return True

def play_game():
    board = [[" " for _ in range(3)] for _ in range(3)]
    players = ["X", "O"]
    current_player = 0

    while True:
        print_board(board)
        player = players[current_player]
        try:
            row = int(input(f"Player {player}, enter row (0-2): "))
            col = int(input(f"Player {player}, enter column (0-2): "))
            if not (0 <= row <= 2 and 0 <= col <= 2):
                print("Invalid input. Row and column must be between 0 and 2.")
                continue
        except ValueError:
            print("Invalid input. Please enter a number.")
            continue

        if board[row][col] == " ":
            board[row][col] = player
            if check_win(board, player):
                print_board(board)
                print(f"Player {player} wins!")
                break
            elif check_draw(board):
                print_board(board)
                print("It's a draw!")
                break
            else:
                current_player = (current_player + 1) % 2
        else:
            print("That spot is already taken! Try again.")

    play_game()

```

```

print("ShashankGowdaL 1BM23CS311")

rooms = {'A': True, 'B': True}

current_room = input("Enter starting room (A/B): ").upper()
while current_room not in rooms:
    current_room = input("Invalid input. Enter starting room (A/B): ").upper()

def suck(room):
    if rooms[room]:
        print(f"Sucking dirt in room {room}.")
        rooms[room] = False
    else:
        print(f"Room {room} is already clean.")

def move(direction):
    global current_room
    if direction == "left" and current_room == "B":
        current_room = "A"
        print("Moved left to room A.")
    elif direction == "right" and current_room == "A":
        current_room = "B"
        print("Moved right to room B.")
    else:
        print("Cannot move in that direction from current room.")

while rooms['A'] or rooms['B']:
    print(f"\nCurrent room: {current_room}")
    print(f"Room A dirty? {rooms['A']}")
    print(f"Room B dirty? {rooms['B']}")
    action = input("Enter action (left, right, suck): ").lower()
    if action == "suck":
        suck(current_room)
    elif action in ["left", "right"]:
        move(action)
    else:
        print("Invalid action. Try again.")

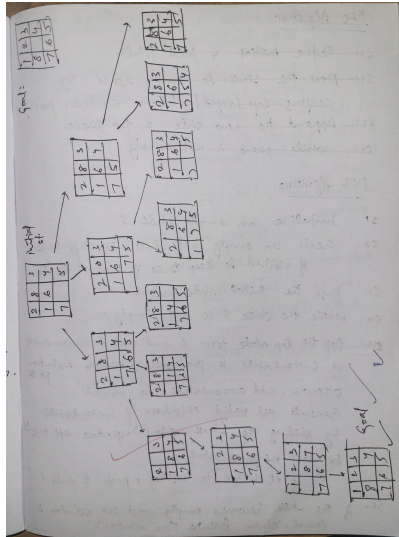
print("Both rooms are clean.")

```

## Program 2

Implement 8 puzzle problems using Depth First Search (DFS)

Implement Iterative deepening search algorithm



DFS Algorithm:

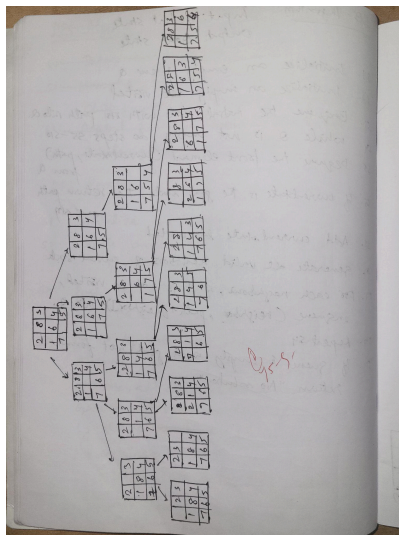
- S1. Define initial & final states
- S2. Move the blocks to empty space, by sliding left/right/up/down whatever possible
- S3. Append the new state in the queue
- S4. while queue is not empty

DFS Algorithm:

- S1. Initialize an empty state S
- S2. Create an empty set visited to keep track of visited & keep track of visited states
- S3. Push the initial state onto S
- S4. while the state S is not empty:
  - pop the top state from S and call it Current state
  - If Current state is goal state, return solution path
  - Otherwise, add current state to visited
  - Generate all valid neighbors of current state by sliding the blank tile (0) up, down, left & right
  - For each neighbor state:
    - If it is not in visited, push it onto S
- S5. If the while becomes empty and no solution found, return failure (no solution)

BFS Algorithm: Input: initial state  
Output: Goal state

- S1. Initialize an empty queue Q
- S2. Initialize an empty set visited
- S3. Enqueue the initial state with its path into Q
- S4. while Q is not empty, do steps S5-S10
- S5. Dequeue the front element (Current state, path) from Q
- S6. If current state is the goal state, then return path and stop
- S7. Add current state to visited
- S8. Generate all valid neighbors of current state
- S9. For each neighbor, if it is not in visited, enqueue (Neighbor, path + [NeighborID]) into Q
- S10. Repeat S4
- S11. If queue is empty and goal not found, return "No solution"



Iterative deepening - DFS:

function IDFS (start, goal):

```

depth = 0
loop:
    result = DFS (start, goal, depth)
    if result == FOUND:
        return "Goal found"
    depth++
  
```

function DFS (node, goal, limit):

```

if node == goal:
    return FOUND
else if limit == 0:
    return NOT found
  
```

Output:

Solution found in 5 moves:

|   |   |   |
|---|---|---|
| 2 | 8 | 3 |
| 1 | 6 | 4 |
| 7 | 5 | 0 |

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 8 | 0 | 4 |
| 7 | 6 | 5 |

```
from collections import deque
```

```
print("Shashank Gowda L, 1BM23CS311\n")
```

```
initial_state = ((2, 8, 3),
                 (1, 6, 4),
                 (7, 0, 5))
```

```
goal_state = ((1, 2, 3),
              (8, 0, 4),
              (7, 6, 5))
```

```
directions = {'Up': (-1, 0), 'Down': (1, 0), 'Left': (0, -1), 'Right': (0,
```

```

1)}

def get_blank_pos(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return (i, j)

def swap_positions(state, pos1, pos2):
    state_list = [list(row) for row in state]
    r1, c1 = pos1
    r2, c2 = pos2
    state_list[r1][c1], state_list[r2][c2] = state_list[r2][c2],
state_list[r1][c1]
    return tuple(tuple(row) for row in state_list)

def get_neighbors(state):
    neighbors = []
    r, c = get_blank_pos(state)
    for move, (dr, dc) in directions.items():
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new_state = swap_positions(state, (r, c), (nr, nc))
            neighbors.append((new_state, move))
    return neighbors

def dfs_8_puzzle(start, goal):
    stack = []
    visited = set()
    visited.add(start)
    stack.append((start, []))

    levels = []

    while stack:
        state, path = stack.pop()
        levels.append(state)

        if state == goal:
            return path, levels, len(visited)

```

```

        for neighbor, move in get_neighbors(state):
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append((neighbor, path + [move]))

    return None, levels, len(visited)

solution_path, level_states, total_visited = dfs_8_puzzle(initial_state,
goal_state)

print(f"Solution length: {len(solution_path)} moves")
print("Solution moves:", solution_path)
print(f"Total states visited: {total_visited}\n")

print("States traversed:")
for i, state in enumerate(level_states):
    print(f"\nState {i+1}:")
    for row in state:
        print(row)
    print("----")

```

```

print("ShashankGowdaL 1BM23CS311")

def goal_state_reached(state, goal_state):
    return state == goal_state

def find_empty(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return (i, j)

def get_neighbors(state):
    neighbors = []
    x, y = find_empty(state)
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]

```



```

        new_state[x][y], new_state[nx][ny] = new_state[nx][ny],
new_state[x][y]
        neighbors.append(new_state)
    return neighbors

def depth_limited_dfs(state, goal_state, depth):
    if depth == 0:
        return [state] if goal_state_reached(state, goal_state) else None
    elif depth > 0:
        for neighbor in get_neighbors(state):
            path = depth_limited_dfs(neighbor, goal_state, depth - 1)
            if path:
                return [state] + path
    return None

def iterative_deepening_dfs(start_state, goal_state):
    depth = 0
    while True:
        path = depth_limited_dfs(start_state, goal_state, depth)
        if path:
            return path
        depth += 1

start_state = [[1, 2, 3],
               [4, 5, 6],
               [7, 8, 0]]

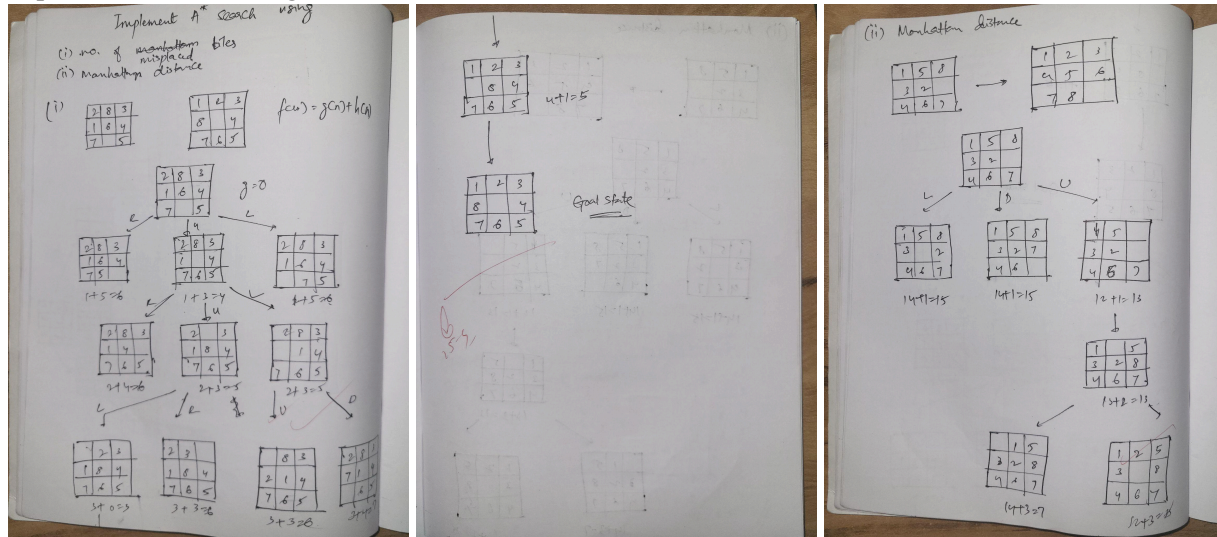
goal_state = [[1, 2, 3],
              [4, 5, 6],
              [7, 8, 0]]

solution = iterative_deepening_dfs(start_state, goal_state)
for step in solution:
    print(step)

```

### Program 3

Implement A\* search algorithm



```
import heapq

print("ShashankGowdaL 1BM23CS311")

goal_state = [[1, 2, 3],
              [4, 5, 6],
              [7, 8, 0]]

def manhattan_distance(state, goal):
    distance = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0:
                goal_x, goal_y = divmod(goal_state[i][j] - 1, 3)
                distance += abs(i - goal_x) + abs(j - goal_y)
    return distance

def find_empty(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return (i, j)

def get_neighbors(state):
    neighbors = []
    x, y = find_empty(state)
```



```

directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
for dx, dy in directions:
    nx, ny = x + dx, y + dy
    if 0 <= nx < 3 and 0 <= ny < 3:
        new_state = [row[:] for row in state]
        new_state[x][y], new_state[nx][ny] = new_state[nx][ny],
new_state[x][y]
        neighbors.append(new_state)
    return neighbors

def a_star(start_state, goal_state):
    open_list = []
    heapq.heappush(open_list, (manhattan_distance(start_state, goal_state), 0,
start_state, []))
    closed_list = set()

    while open_list:
        _, g, current_state, path = heapq.heappop(open_list)
        if current_state == goal_state:
            return path + [current_state]

        closed_list.add(tuple(map(tuple, current_state)))

        for neighbor in get_neighbors(current_state):
            if tuple(map(tuple, neighbor)) not in closed_list:
                f = g + 1 + manhattan_distance(neighbor, goal_state)
                heapq.heappush(open_list, (f, g + 1, neighbor, path +
[current_state]))

    return None

start_state = [[1, 2, 3],
               [4, 5, 6],
               [7, 8, 0]]

solution = a_star(start_state, goal_state)
for step in solution:
    print(step)

```

```

import heapq

print("ShashankGowdaL 1BM23CS311")
goal_state = [[1, 2, 3],
               [4, 5, 6],
               [7, 8, 0]]

def misplaced_tiles(state, goal):
    count = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != goal[i][j] and state[i][j] != 0:
                count += 1
    return count

def find_empty(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return (i, j)

def get_neighbors(state):
    neighbors = []
    x, y = find_empty(state)
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in directions:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [row[:] for row in state]
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny],
new_state[x][y]
            neighbors.append(new_state)
    return neighbors

def a_star(start_state, goal_state):
    open_list = []
    heapq.heappush(open_list, (misplaced_tiles(start_state, goal_state), 0,
start_state, []))
    closed_list = set()

    while open_list:

```

```

_, g, current_state, path = heapq.heappop(open_list)
if current_state == goal_state:
    return path + [current_state]

closed_list.add(tuple(map(tuple, current_state)))

for neighbor in get_neighbors(current_state):
    if tuple(map(tuple, neighbor)) not in closed_list:
        f = g + 1 + misplaced_tiles(neighbor, goal_state)
        heapq.heappush(open_list, (f, g + 1, neighbor, path +
[current_state]))

return None

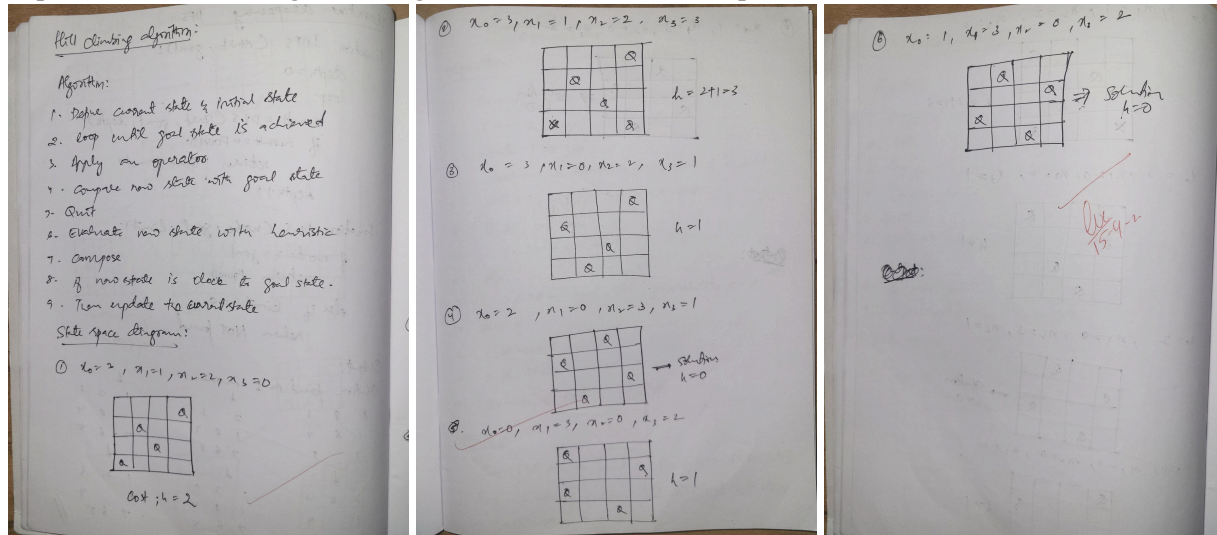
start_state = [[1, 2, 3],
               [4, 5, 6],
               [7, 8, 0]]

solution = a_star(start_state, goal_state)
for step in solution:
    print(step)

```

## Program 4

Implement Hill Climbing search algorithm to solve N-Queens problem



```
import random
```

```
def create_board():
```

```
    return [random.randint(0, 3) for _ in range(4)]
```

```
def calculate_conflicts(board):
```

```
    conflicts = 0
```

```
    for i in range(4):
```

```
        for j in range(i + 1, 4):
```

```
            if board[i] == board[j] or abs(board[i] - board[j]) == j - i:
```

```
                conflicts += 1
```

```
    return conflicts
```

```
def hill_climbing():
```

```
    board = create_board()
```

```
    print(f'Initial board: {board} with conflicts: {calculate_conflicts(board)}')
```

```
    while True:
```

```
        current_conflicts = calculate_conflicts(board)
```

```
        if current_conflicts == 0:
```

```
            return board
```

```
        next_board = None
```

```
        next_conflicts = float('inf')
```

```
        for i in range(4):
```

```
            temp_board = board[:]
```

```
            for j in range(4):
```

```
                if temp_board[i] != j:
```

```
                    temp_board[i] = j
```

```
                    temp_conflicts = calculate_conflicts(temp_board)
```

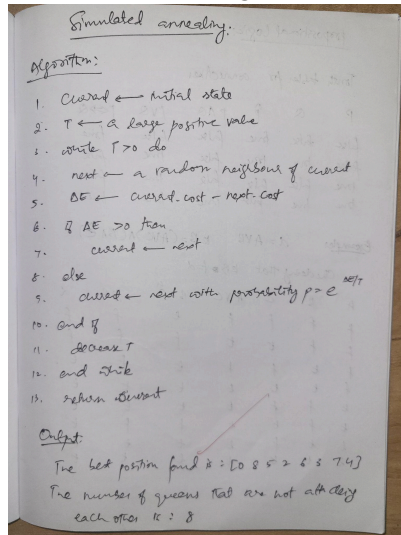
```
                    print(f'Board: {temp_board} with conflicts: {temp_conflicts}')
```

```
        if temp_conflicts < next_conflicts:
            next_conflicts = temp_conflicts
            next_board = temp_board[:]
    if next_conflicts >= current_conflicts:
        return board
    board = next_board

solution = hill_climbing()
print(f"Final solution: {solution}")
```

## Program 5

### Simulated Annealing to Solve 8-Queens problem



```
import random
import math

print("ShashankGowdaL 1BM23CS311")

def objective_function(state):
    return sum(state)

def generate_neighbor(state):
    neighbor = state[:]
    index = random.randint(0, len(state) - 1)
    neighbor[index] = random.randint(1, 100)
    return neighbor

def simulated_annealing(initial_state, initial_temp, cooling_rate,
iterations):
    current_state = initial_state
    current_temp = initial_temp
    best_state = current_state
    best_score = objective_function(current_state)

    for i in range(iterations):
        neighbor = generate_neighbor(current_state)
        current_score = objective_function(current_state)
        neighbor_score = objective_function(neighbor)
        if neighbor_score < best_score:
```

```

        best_state = neighbor
        best_score = neighbor_score

        if neighbor_score < current_score or random.random() <
math.exp((current_score - neighbor_score) / current_temp):
            current_state = neighbor

        current_temp *= cooling_rate

    return best_state, best_score

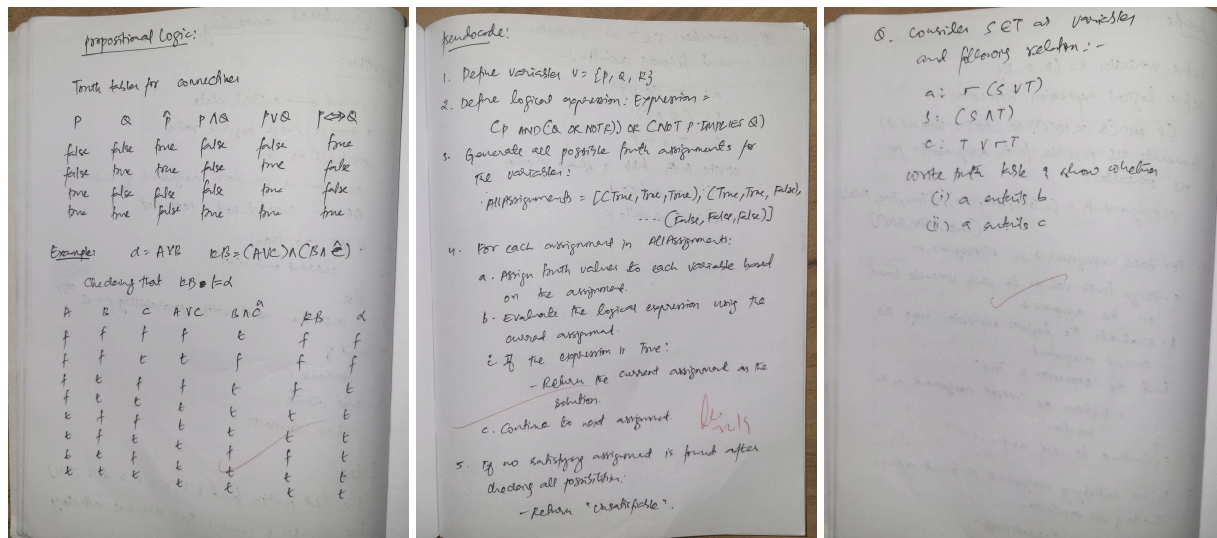
initial_state = [random.randint(1, 100) for _ in range(10)]
initial_temp = 1000
cooling_rate = 0.99
iterations = 1000

best_state, best_score = simulated_annealing(initial_state, initial_temp,
cooling_rate, iterations)
print("Best State:", best_state)
print("Best Score:", best_score)

```

## Program 6

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.



```
import itertools
```

```
def evaluate_expression(expression, assignment):
```

```
    expression = expression.replace('A', str(assignment[0]))
```

```
    expression = expression.replace('B', str(assignment[1]))
```

```
    expression = expression.replace('C', str(assignment[2]))
```

```
    expression = expression.replace('and', 'and').replace('or', 'or').replace('not', 'not')
```

```
    return eval(expression)
```

```
def check_ entailment(alpha, kb):
```

```
    variables = ['A', 'B', 'C']
```

```
    all_assignments = list(itertools.product([False, True], repeat=len(variables)))
```

```
    print("Truth Table:")
```

```
    print("A\tB\tC\tKB\tα")
```

```
    for assignment in all_assignments:
```

```
        kb_result = evaluate_expression(kb, assignment)
```

```
        alpha_result = evaluate_expression(alpha, assignment)
```

```
    print(f'{assignment[0]}\t{assignment[1]}\t{assignment[2]}\t{kb_result}\t{alpha_result}')
```

```
    if kb_result and not alpha_result:
```

```
        return False
```

```
    return True
```

```
alpha = "A or B"
```

```
kb = "(A or C) and (B and not C)"
```



```
if check_entailment(alpha, kb):  
    print("KB entails  $\alpha$ ")  
else:  
    print("KB does not entail  $\alpha$ ")
```

## Program 7

Implement unification in first order logic.

Unification Algorithm:  
 Unification is the process to find substitution that make different first order logic identity

Ex: Unify (knows(John, x), knows(John, Jane))  
 $\theta = x/Jane$   
 Unify (knows(John, Jane), knows(John, Jane))  
 True

Ex: Find MGO of  
 $\{ \{ p(C_1, x, f(g(x))), \{ p(C_2, f(x), f(x)) \} \}$   
 $\{ p(C_3, f(x), f(x)) \}$  True

Ex: Find MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, f(x), f(x)) \}$   
 $\theta = \{ x/Jane, \theta = x/f(x) \}$  True

Ex: Find MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, f(x), f(x)) \}$   
 $\theta = \{ x/f(x), \theta = x/f(x) \}$  True

Ex: Find MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, f(x), f(x)) \}$   
 $\theta = \{ x/f(x), \theta = x/f(x) \}$  True

Ex: Find the MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, x, f(x)) \}$   
 $\theta = x/f(x)$  True

Ex: Find the MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, x, f(x)) \}$   
 $\theta = x/f(x)$  True

Ex: Find the MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, x, f(x)) \}$   
 $\theta = x/f(x)$  True

Ex: Find the MGO of  $\{ p(C_1, x, f(g(x))), p(C_2, x, f(x)) \}$   
 $\theta = x/f(x)$  True

Algorithm: Unify( $P_1, P_2$ )

Step1: If  $P_1$  or  $P_2$  is a variable or constant, then:  
 a) If  $P_1$  or  $P_2$  are identical, then return TRUE  
 b) Else if  $P_1$  is a variable,  
 a. Then if  $P_1$  occurs in  $P_2$ , then return FAILURE  
 c. Else return  $\{ C_1/P_2 \}$   
 d) Else if  $P_2$  is a variable,  
 a. If  $P_2$  occurs in  $P_1$  then return FAILURE  
 b. Else return  $\{ C_2/P_1 \}$   
 c) Else return FAILURE

Step2: If the initial predicate symbol in  $P_1$  and  $P_2$  are not same, then return FAILURE

Step3: If  $P_1$  and  $P_2$  have a different number of arguments, then return FAILURE

Step4: Set substitution set SUBST to NIL

Step5: For  $i = 1$  to the number of elements in  $P_1$ :  
 a) Call Unify function with the  $i$ th element of  $P_1$  and its element of  $P_2$ , and put the result into SUBST  
 b) If SUBST is failure then return FAILURE

Ex: If  $S \neq NIL$ , then do,  
 a. Apply  $S$  to the remainder of  $C_1$  and  $C_2$   
 b. SUBST = APPEND( $S, SUBST$ )

Steps: return SUBST

Output:

```
print("Shashank Gowda 1BM23CS311")
```

```
class Term:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def __repr__(self):
```

```
        return self.name
```

```
class Variable(Term):
```

```
    def __init__(self, name):
```

```
        super().__init__(name)
```

```
class Constant(Term):
```

```

def __init__(self, name):
    super().__init__(name)

class Predicate:
    def __init__(self, symbol, terms):
        self.symbol = symbol
        self.terms = terms

    def __repr__(self):
        return f"{self.symbol}({' , '.join(map(str, self.terms))})"

def unify(x, y, subst):
    if subst is None:
        return None

    if x == y:
        return subst

    elif isinstance(x, Variable) and not isinstance(y, Variable):
        return unify_var(x, y, subst)

    elif isinstance(y, Variable) and not isinstance(x, Variable):
        return unify_var(y, x, subst)

    elif isinstance(x, Predicate) and isinstance(y, Predicate):
        if x.symbol != y.symbol:
            raise Exception(f"Cannot unify predicates with different symbols:
{x} and {y}")
        else:
            return unify_lists(x.terms, y.terms, subst)

    else:
        raise Exception(f"Unification failed: {x} and {y} are not unifiable")

def unify_var(var, term, subst):
    if var in subst:
        return unify(subst[var], term, subst)

    if occurs_check(var, term):
        raise Exception(f"Occurs check failed: {var} cannot be unified with
{term}")

```

```

    subst[var] = term
    return subst

def occurs_check(var, term):
    if isinstance(term, Variable):
        return term == var
    elif isinstance(term, Predicate):
        return any(occurs_check(var, t) for t in term.terms)
    return False

def unify_lists(list_x, list_y, subst):
    if len(list_x) != len(list_y):
        raise Exception(f"Cannot unify lists of different lengths: {list_x}
and {list_y}")

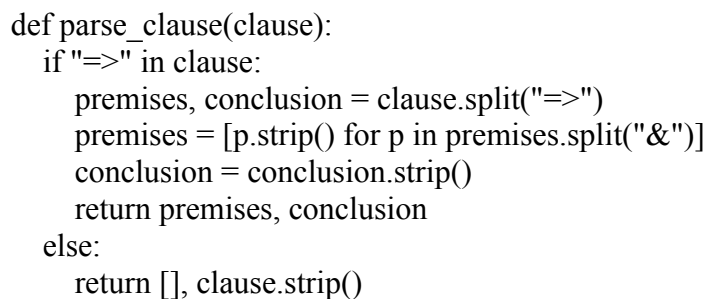
    for term_x, term_y in zip(list_x, list_y):
        subst = unify(term_x, term_y, subst)
        if subst is None:
            return None
    return subst

if __name__ == "__main__":
    x = Variable('X')
    y = Variable('Y')
    c = Constant('a')
    p1 = Predicate('father', [x, c])
    p2 = Predicate('father', [y, c])

    substitution = unify(p1, p2, {})
    if substitution is not None:
        print("Unification succeeded:", substitution)
    else:
        print("Unification failed.")

```

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.



```

added = True
while added:
    added = False
    for premises, conclusion in rules:
        if all(p in known for p in premises) and conclusion not in known:
            known.add(conclusion)
            added = True
    if query in known:
        return True
return query in known

```

```
n = int(input("Enter number of statements in KB: "))
kb = [input(f"Statement {i+1}: ") for i in range(n)]
query = input("Enter query: ")

if forward_reasoning(kb, query):
    print("Query is proved by forward reasoning")
else:
    print("Query cannot be proved")
```



## Program 9

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution.

Resolution in First Order Logic

Algorithm:

- Eliminate biconditionals and implications  
 Eliminate  $\leftrightarrow$ , replacing  $A \leftrightarrow B$  with  $(A \supset B) \wedge (B \supset A)$   
 Eliminate  $\rightarrow$ , replacing  $A \rightarrow B$  with  $\neg A \vee B$
- Move  $\neg$  inwards  
 $\neg(\neg A) \equiv A$

Steps for resolution:

- Convert all sentences to CNF
- Negate conclusion  $S$  & convert premise to CNF
- Add negated conclusion  $S$  to the premise clauses
- Repeat until contradiction or no program is made:
  - Select a clause (avoid those with constants)
  - Resolve from top down, performing all required substitutions
  - If resultant is the empty clause, a contradiction has been found (i.e.  $S$  follows from the premises)
  - If not, add resultant to the premises

If we succeed in step 4, we have proved the conclusion

Problem:

Representation in FOL:

- $\forall x: \text{food}(x) \rightarrow \text{likes}(\text{john}, x)$
- $\text{food}(\text{apple}) \wedge \text{food}(\text{vegetables})$
- $\forall x, y: \text{eats}(x, y) \wedge \neg \text{bitled}(x) \rightarrow \text{food}(y)$
- $\text{eats}(\text{chris}, \text{peanuts}) \wedge \text{alive}(\text{chris})$
- $\text{tho} \rightarrow \text{eats}(\text{chris}, x) \rightarrow \text{eats}(\text{tho}, x)$
- $\text{tho} \rightarrow \text{bitled}(x) \rightarrow \text{alive}(x)$
- $\text{tho} \rightarrow \text{alive}(x) \rightarrow \text{bitled}(x)$
- $\text{likes}(\text{john}, \text{peanuts})$

Eliminate implications:

- $\neg \text{tho} \rightarrow \text{food}(x) \vee \text{likes}(\text{john}, x)$
- $\text{food}(\text{apple}) \wedge \text{food}(\text{vegetables})$
- $\text{tho} \vee \neg \text{eats}(x, y) \vee \neg \text{bitled}(x) \vee \text{food}(y)$
- $\text{eats}(\text{chris}, \text{peanuts}) \wedge \text{alive}(\text{chris})$
- $\text{tho} \rightarrow \text{eats}(\text{chris}, x) \vee \text{eats}(\text{tho}, x)$
- $\neg \text{tho} \vee \text{bitled}(x) \vee \neg \text{alive}(x)$
- $\text{tho} \rightarrow \text{alive}(x) \vee \neg \text{bitled}(x)$
- $\text{likes}(\text{john}, \text{peanuts})$

More negation inwards & rename:

- $\neg \text{tho} \rightarrow \text{food}(x) \vee \text{likes}(\text{john}, x)$
- $\text{food}(\text{apple}) \wedge \text{food}(\text{vegetables})$
- $\text{tho} \vee \neg \text{eats}(x, y) \vee \neg \text{bitled}(x) \vee \text{food}(y)$
- $\text{eats}(\text{chris}, \text{peanuts}) \wedge \text{alive}(\text{chris})$
- $\neg \text{tho} \vee \text{eats}(\text{chris}, x) \vee \text{eats}(\text{tho}, x)$
- $\neg \text{tho} \vee \text{bitled}(x) \vee \neg \text{alive}(x)$
- $\text{tho} \rightarrow \text{alive}(x) \vee \neg \text{bitled}(x)$
- $\text{likes}(\text{john}, \text{peanuts})$

Drop constants:

- $\neg \text{tho} \vee \text{food}(x) \vee \text{likes}(\text{john}, x)$
- $\text{food}(\text{apple})$
- $\text{food}(\text{vegetables})$
- $\text{tho} \vee \neg \text{eats}(y, z) \vee \neg \text{bitled}(y) \vee \text{food}(z)$
- $\text{eats}(\text{chris}, \text{peanuts})$
- $\text{alive}(\text{chris})$
- $\neg \text{tho} \vee \text{eats}(\text{chris}, x) \vee \text{eats}(\text{tho}, x)$
- $\neg \text{tho} \vee \text{bitled}(x) \vee \neg \text{alive}(x)$
- $\text{likes}(\text{john}, \text{peanuts})$

Resolution:

- $\neg \text{tho} \vee \text{food}(x) \vee \text{likes}(\text{john}, x)$
- $\text{food}(\text{apple})$
- $\text{food}(\text{vegetables})$
- $\text{tho} \vee \neg \text{eats}(y, z) \vee \neg \text{bitled}(y) \vee \text{food}(z)$
- $\text{eats}(\text{chris}, \text{peanuts})$
- $\text{alive}(\text{chris})$
- $\neg \text{tho} \vee \text{eats}(\text{chris}, x) \vee \text{eats}(\text{tho}, x)$
- $\neg \text{tho} \vee \text{bitled}(x) \vee \neg \text{alive}(x)$
- $\text{likes}(\text{john}, \text{peanuts})$

Resolution tree:

```

graph TD
    A["¬likes(john, peanuts) → food(x) ∨ likes(john, x)"] --> B["¬food(peanuts)"]
    A --> C["food(x) ∨ likes(john, x)"]
    B --> D["¬eats(y, z) ∨ bitled(y) ∨ food(z)"]
    B --> E["eats(chris, peanuts)"]
    D --> F["bitled(chris)"]
    D --> G["¬bitled(chris)"]
    F --> H["¬alive(chris)"]
    F --> I["alive(chris)"]
    G --> J["¬alive(chris)"]
    G --> K["alive(chris)"]
    H --> L["Hence proved"]
    I --> L
    J --> L
    K --> L
  
```

o/p

```
from sympy import symbols, Or, Not
```

```
P, Q, R = symbols('P Q R')
```

```
kb = [
```

```
    (Not(P), Q),
```

```
    (Not(Q), R),
```

```
    (P,)
]
```

```
query = R
```

```
def resolve(ci, cj):
```

```
    resolvents = set()
```

```

    for di in ci:
        for dj in cj:
            if di == Not(dj) or Not(di) == dj:
                new_clause = tuple(sorted(set(ci + cj) - {di, dj}, key=str))
                if not new_clause:
                    resolvents.add(())
                else:
                    resolvents.add(new_clause)
    return resolvents

def pl_resolution(kb, query):
    clauses = kb + [(Not(query),)]
    new = set()
    while True:
        pairs = [(clauses[i], clauses[j]) for i in range(len(clauses)) for j
in range(i+1, len(clauses))]
        for (ci, cj) in pairs:
            resolvents = resolve(ci, cj)
            if () in resolvents:
                return True
            new.update(resolvents)
        if new.issubset(set(clauses)):
            return False
        for c in new:
            if c not in clauses:
                clauses.append(c)

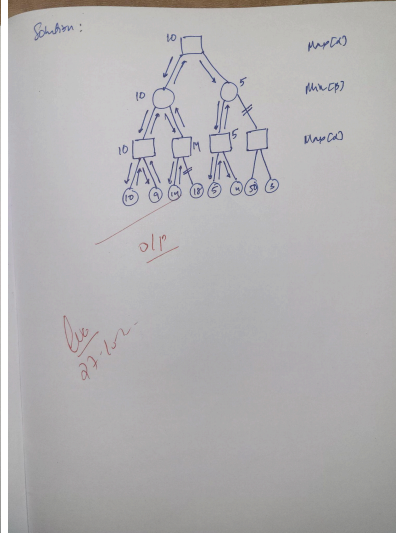
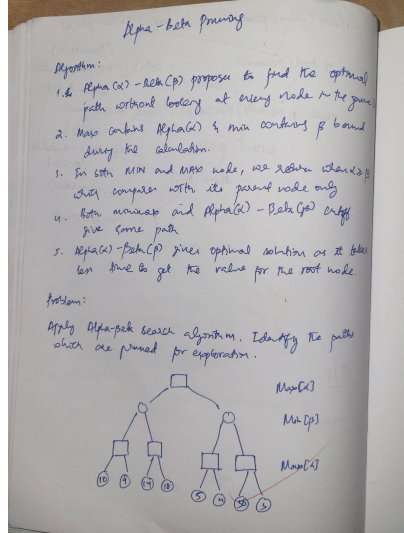
print(pl_resolution(kb, query))

```



## Program 10

### Implement Alpha-Beta Pruning.



```
import math

def alpha_beta(depth, node_index, is_maximizing, values, alpha, beta,
max_depth):
    indent = "  " * depth # indentation for visualization
    if depth == max_depth:
        print(f"{indent}Leaf node[{node_index}] = {values[node_index]}")
        return values[node_index]

    if is_maximizing:
        best = -math.inf
        print(f"{indent}MAX node ( $\alpha$ ={alpha},  $\beta$ ={beta})")
        for i in range(2):
            val = alpha_beta(depth + 1, node_index * 2 + i, False, values,
alpha, beta, max_depth)
            best = max(best, val)
            alpha = max(alpha, best)
            print(f"{indent}  MAX updating  $\alpha$ ={alpha}")
            if beta <= alpha:
                print(f"{indent}   $\beta$  cut-off ( $\beta$ ={beta},  $\alpha$ ={alpha}) ")
                break
        return best
    else:
        best = math.inf
        print(f"{indent}MIN node ( $\alpha$ ={alpha},  $\beta$ ={beta})")
```

```

        for i in range(2):
            val = alpha_beta(depth + 1, node_index * 2 + i, True, values,
alpha, beta, max_depth)
            best = min(best, val)
            beta = min(beta, best)
            print(f"{indent}    MIN updating  $\beta$ = {beta}")
            if beta <= alpha:
                print(f"{indent}     $\alpha$  cut-off ( $\beta$ = {beta},  $\alpha$ = {alpha}) ")
                break
        return best

# Example
values = [10, 9, 14, 18, 5, 4, 50, 3]
max_depth = int(math.log2(len(values)))

print("Leaf Nodes:", values)
result = alpha_beta(0, 0, True, values, -math.inf, math.inf, max_depth)
print("\nOptimal value:", result)

```